Python

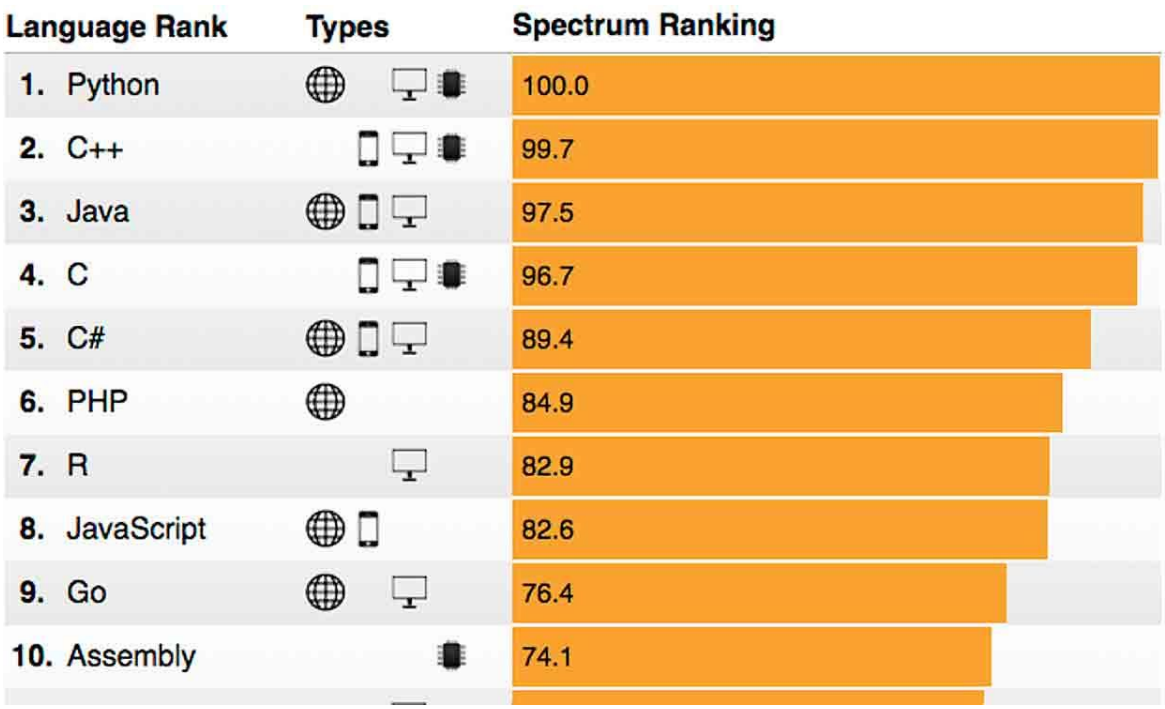
Big Data Analytics

Jong Hwan Suh

Lecture 2

Python Popularity

- Top Ten Languages of 2018, as ranked for the typical IEEE member and *Spectrum* reader.
  - https://spectrum.ieee.org/at-work/innovation/the-2018-top-programming-languages

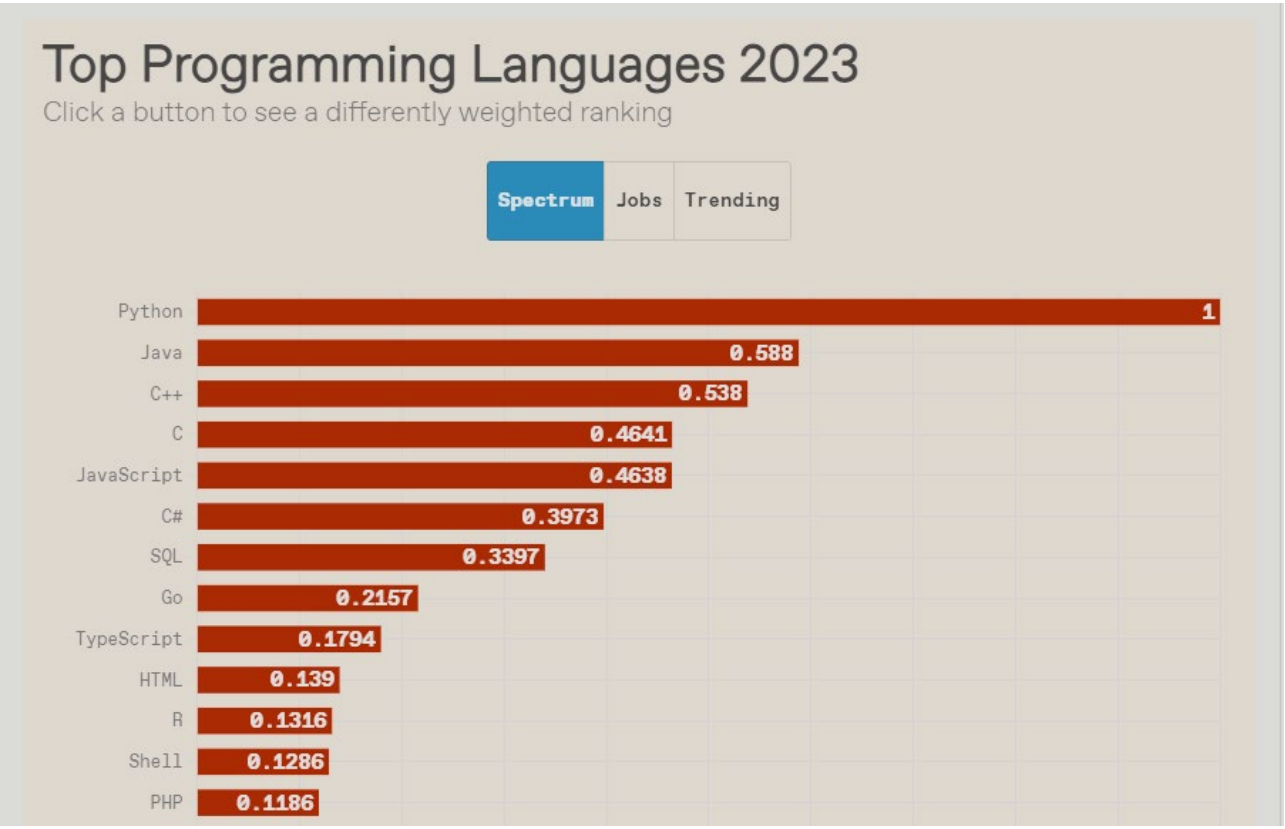


Python Popularity

- Top Ten Languages of 2022, as ranked for the typical IEEE member and *Spectrum* reader.
  - https://spectrum.ieee.org/top-programming-languages-2022

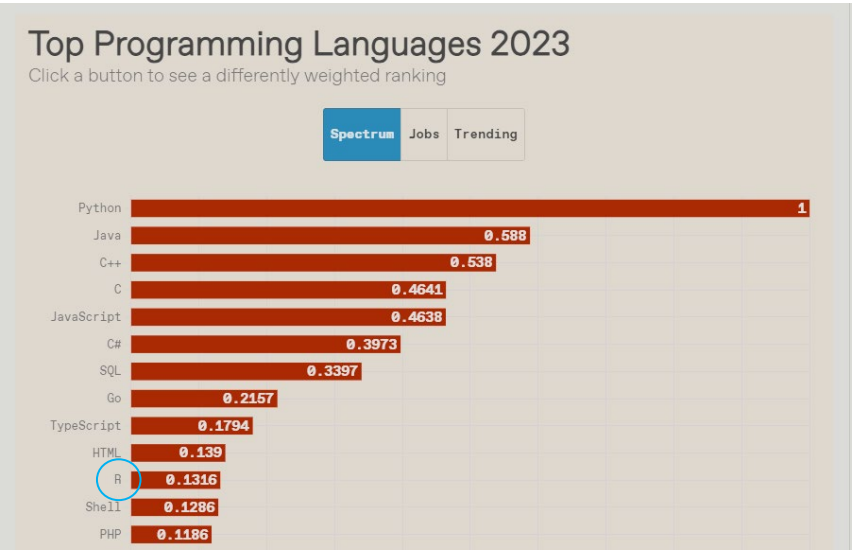
Python Popularity

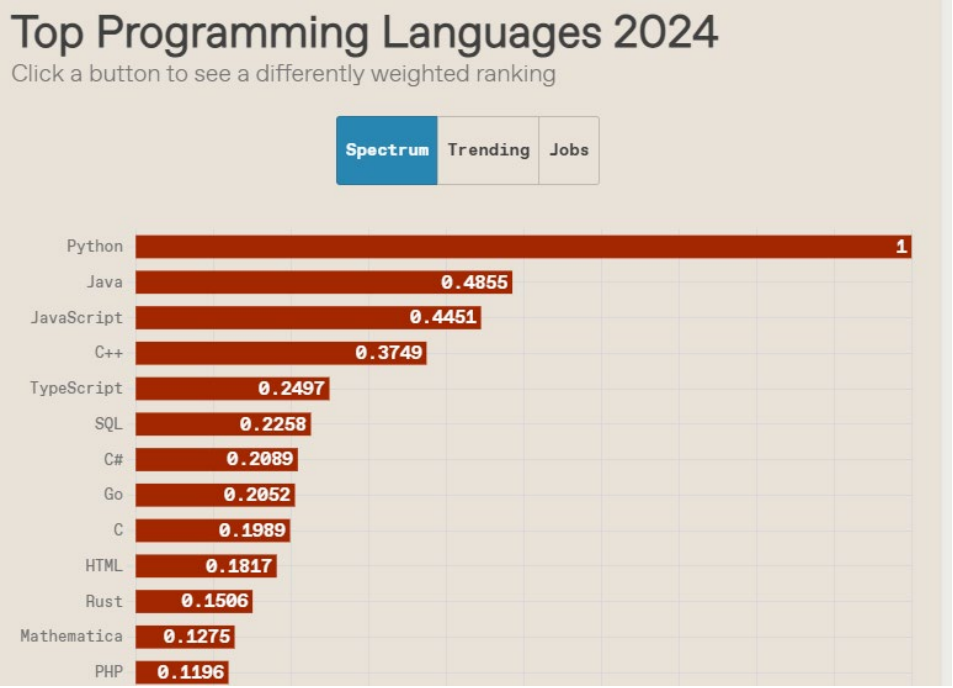
- Top Ten Languages of 2023, as ranked for the typical IEEE member and *Spectrum* reader.
  - https://spectrum.ieee.org/the-top-programming-languages-2023



Python Popularity

- Top Ten Languages of 2024, as ranked for the typical IEEE member and *Spectrum* reader.
  - https://spectrum.ieee.org/top-programming-languages-2024





Python Popularity

- Top Ten Languages of 2025, as ranked for the typical IEEE member and *Spectrum* reader.
  - https://spectrum.ieee.org/top-programming-languages-2025

Python is Simple

- The simplicity is one of the reason that Python is a language of choice.

C

```
#include "stdio.h"
int main() {
  printf("Hello\n");
}
```

python

```
print("hello")
```

Java

```
public class Hi {
  public static void main (String [] args) {
    System.out.println("Hello");
  }
}
```

Python is Simple

- Python does not have ';'(semicolon).

C

```
#include "stdio.h"
int main() {
  printf("Hello\n");
}
```

python

```
print("hello")
```

Java

```
public class Hi {
  public static void main (String [] args) {
    System.out.println("Hello");
  }
}
```

Python is Simple

- Python does not have to declare types.

C

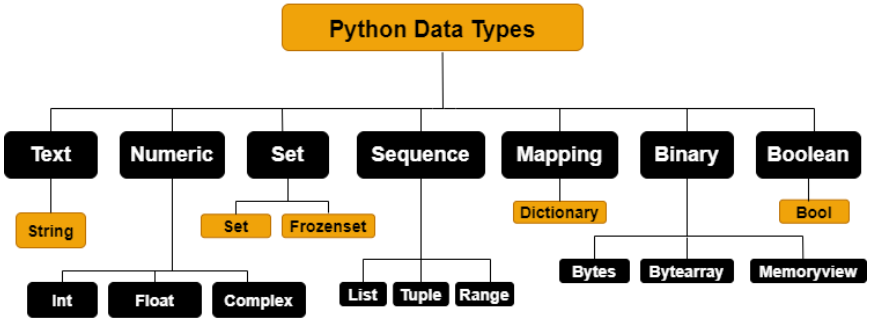
```
#include "stdio.h"

int main() {
  int x = 3;
  int y = 4;
  printf("%s"\n,x+y);
}
```

python

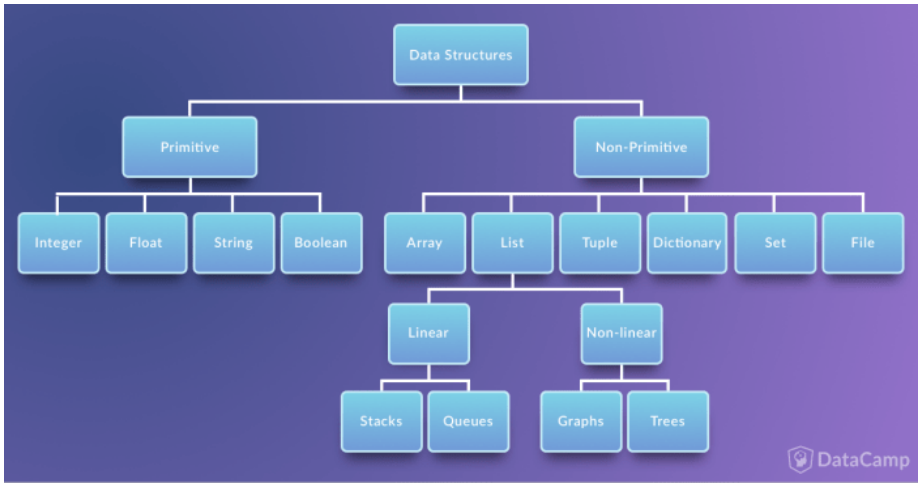
```
x = 3
y = 4
print(x+y)
```

Variables, Data Types

- Text: string
- Numeric: integers, float, complex
- Set: set
- Sequence: list, tuple, range
- Mapping: dict
- Binary: byte, bytearray
- Boolean(True/False): bool



https://www.lore-hub.com/2020/09/data-types-in-python.html



https://www.datacamp.com/community/tutorials/data-structures-python

Dynamic Typing

C

```c
#include "stdio.h"

int main() {
  int x = 3;
  x = 4.5;
}
```

python

```python
x = 3
x = 4.5
```

Everything in Python is an Object

- x is being created on the stack, and the PyIntObject with the value of three is being created on the heap.
- The heap holds dynamically created data and is actually managed by the operating system.

```
>>> x = 3
>>> y = 3.0
>>> x is y
```

```
>>> x = 3
>>> y = 3.0
>>> x == y
True
```

Everything in Python is an Object

```
>>> x = "Hello"
>>> x = x.lower()
>>> x
'hello'
```

Everything in Python is an Object / Labs

- x is being created on the stack, and the PyIntObject with the value of three is being created on the heap.
- The heap holds dynamically created data and is actually managed by the operating system.

```
>>> x = 3
>>> y = 3.0
>>> x is y
```

```
>>> x = 3
>>> y = 3.0
>>> x == y
True
```

Loops / Labs

- Indenting in Python is the same as putting a block of code in brackets in C++ or Java.
- Range returns the values between start and stop (exclusive), increasing by the value of step (defaults to 1).

```
for i in range(0,10, 2):
    print(i)
```

Loops / Labs

- while loop, same with the for loop:

```
i = 2
while i < 12:
    print(i)
    i+=3
```

Conditions / Labs

- % (modulo) represents x % y produces the remainder from x / y.
- In condition line, everything after : (colon), which is indented, will be executed if the condition is false.

```
for i in range(0,5):
    if i % 3 == 0:
        print(i)
    elif i % 3 == 1:
        print(i+10)
    else:
        print(i-10)
```

Functions

- Compared to C code, neither return type nor the type of input parameters are defined.

C

```
int my_abs( int val) {
  if(val < 0) {
     return 0-val;
  }
  return val;
}
```

python

```
def my_abs(val):
     if val < 0:
          return 0-val
     return val
```

Functions / Labs

- my_abs (-7)
- my_abs (“Hi”)
- my_abs(x)

Functions / Labs

- ```
  def inc_val(val):
      val = val + 1
      return val
  x = 7
  inc_val(x)
  print(x)
  ```
  - val is just a copy of the pointer which means that it points to the same thing as x.
  - Now, val is assigned to be val + 1, and this operation will cause val to be reassigned to point to a new PyIntObject with a value of 8.
  - Hence, x is not going to be changed at all.
  - When the function finishes, the temporary(local) variable goes away.
  ```

Scope

- The variable of the function is not alive out of the function.
  - For that purpose, a global variable should be declared.
- Do not let the global variable have the same name with a function parameter.

```
def my_abs(val):
      if val < 0:
            return 0-val
      return val
```

String

- For more information, visit: https://docs.python.org/2/library/string.html

Python » English 2.7.15 Documentation » The Python Standard Library » 7. String Services »

Table Of Contents

7.1. string — Common string operations
- 7.1.1. String constants
- 7.1.2. Custom String Formatting
- 7.1.3. Format String Syntax
  - 7.1.3.1. Format Specification Mini-Language
  - 7.1.3.2. Format examples
- 7.1.4. Template strings
- 7.1.5. String functions
- 7.1.6. Deprecated string functions

Previous topic

7. String Services

Next topic

7.2. re — Regular expression operations

This Page

Show Source

Quick search

Go

7.1. string — Common string operations

Source code: Lib/string.py

The string module contains a number of useful constants and classes, as well as some deprecated legacy functions that are also string classes support the sequence type methods described in the Sequence Types — str, unicode, list, tuple, bytearray, buffer, xra the String Methods section. To output formatted strings use template strings or the % operator described in the String Formatting O based on regular expressions.

7.1.1. String constants

The constants defined in this module are:

string.**ascii_letters**
The concatenation of the ascii_lowercase and ascii_uppercase constants described below. This value is not locale-dependent.

string.**ascii_lowercase**
The lowercase letters 'abcdefghijklmnopqrstuvwxyz'. This value is not locale-dependent and will not change.

string.**ascii_uppercase**
The uppercase letters 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'. This value is not locale-dependent and will not change.

string.**digits**
The string '0123456789'.

string.**hexdigits**
The string '0123456789abcdefABCDEF'.

string.**letters**
The concatenation of the strings lowercase and uppercase described below. The specific value is locale-dependent, and will be upda

String: Change Case

- Strings are immutable, so string functions return new strings.

```
>>> word = 'Hello'
>>> word.lower()
```

```
hello
```

```
>>> word.upper()
```

```
HELLO
```

String: Concatenation

```
>>> '1' + '2'
```

```
'12'
```

String: Replication

```
>>> '12'*2
```

```
'1212'
```

```
>>> '1'*2 + '2'*3
```

```
'11222'
```

String: Strip

- Returns a copy of the string with leading and trailing characters removed.
- If an input parameter is omitted or none, like, string(), white space characters are removed.

```
>>> s= '   Extras \n'
>>> s.strip()
```

```
'Extras'
```

```
>>> s = '***10***'
>>> s.strip('*')
```

```
'10'
```

String: Split

- Returns a list of the words of the input string.

```
>>> s = 'Let\'s split the words'
>>> s.split(' ')
```

```
["Let's", 'split', 'the', 'words']
```

```
>>> s = 'Jane,Doe,Cars,5'
>>> s.split(',')
```

```
['Jane', 'Doe', 'Cars', '5']
```

String: Slicing / Labs

```
>>> word = 'Hello'
>>> word[1:3]
```

```
'el'
```

```
>>> word[4:7]
```

```
'o'
```

```
>>> word[-4:-1]
```

```
'ell'
```

String: Substring Testing

- find() returns the lowest index in the string where the substring is found. If not found, it returns -1.

```
>>> word = 'Hello'
>>> 'HE' in word
```

```
False
```

```
>>> 'He' in word
```

```
True
```

```
>>> word.find('el')
```

```
1
```

String: Convert to Number

```
>>> word = '1234'
>>> int(word)
```

```
1234
```

```
>>> float(word)
```

```
1234.0
```

```
>>> word = 'Hi'
>>> int(word)
```

< Error >

String: String Formatting

```
>>> statement = 'We love {} {}.'
>>> statement.format('data','analysis')
```

```
'We love data analysis.'
```

```
>>> statement = 'We love {0} {1}.'
>>> statement.format('data','analysis')
```

```
'We love data analysis.'
```

```
>>> statement = 'We love {1} {0}.'
>>> statement.format('analysis','data')
```

```
'We love data analysis.'
```

List: Basics

Index: 0 1 2

```
>>> list = [11,22,33]
>>> list
```

```
[11, 22, 33]
```

```
>>> list[1]
```

```
22
```

```
>>> list[3]
```

Error – index out of range

List: Iterating over a List / Labs

```
>>> list = [11,22,33]
```

```
>>> for i in list:
…       print(i)
```

```
11
22
33
```

```
>>> for i in range(0,len(list)):
…       print(list[i])
```

```
11
22
33
```

List: Mutable

```
>>> list = [11,22,33]
>>> list[1]=95
```

```
>>> print(list[1])
```

```
95
```

List: Appending to a List

```
>>> list = [11,22,33]
```

```
>>> list.append(44)
```

```
>>> list
```

```
[11, 22, 33, 44]
```

List: Deleting from a List

```
>>> list = [11,22,33,44]
>>> list.pop(2)
```

```
33
```

```
>>> list
```

```
[11, 22, 44]
```

List: Removing from a List

```
>>> list = [11,22,33,44]
>>> list.remove(33)
```

```
>>> list
```

```
[11, 22, 44]
```

List: Extending by adding a List to a List

```
>>> list = [1, 2, 3]
>>> list2 = [4, 5, 6]
```

```
>>> list.extend(list2)
>>> list
```

```
[1, 2, 3, 4, 5, 6]
```

```
>>> list = [1, 2, 3]
>>> list2 = [4, 5, 6]
>>> list.append(list2)
>>> list
```

```
[1, 2, 3, [4, 5, 6]]
```

More on Lists

- https://docs.python.org/3/tutorial/datastructures.html#more-on-lists

Tuples: Basics

- Tuples are used to hold multiple types of values.

```
>>> tuple1 = ('Honda','Civic',4,2017)
>>> tuple1
```

```
('Honda', 'Civic', 4, 2017)
```

```
>>> tuple1[1]
```

```
'Civic'
```

```
>>> len(tuple1)
```

```
4
```

Tuples: Iterating over a Tuple

```
>>> tuple1 = ('Honda','Civic',4,2017)
>>> for i in tuple1:
…       print(i)
```

```
Honda
Civic
4
2017
```

Tuples: Immutability

- Once defined, it is never changed.

```
>>> tuple1 = ('Honda','Civic',4,2017)
>>> tuple1[3]=2018
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Dictionary: Basics

- It is used to store key-value pairs, i.e. 1:1, 1:N, and N:1.

```
>>> dict = {('Ghostbusters',2016): 5.4,
('Ghostbusters',1984):7.8}
>>> tuple1
```

```
{('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8}
```

```
>>>  dict[('Ghostbusters',2016)]
```

```
5.4
```

```
>>> len(dict)
```

```
2
```

Dictionary: Adding to a Dictionary

- It is an unordered collection.

```
>>> dict = {('Ghostbusters',2016): 5.4,
('Ghostbusters',1984):7.8}
>>> dict[('Cars',2006)] = 7.1
```

```
>>>  dict
```

```
{('Ghostbusters', 2016): 5.4,
('Cars', 2006): 7.1,
('Ghostbusters', 1984): 7.8}
```

Dictionary: Getting a value from a Dictionary

```
>>> dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars', 2006):7.1}
>>> x = dict[('Cars',2006)]
>>> x
```

```
7.1
```

```
>>> x = dict[('Toy Story',1995)]
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in
<module>
KeyError: ('Cars', 2000)
```

Dictionary: Safer way to get from a dictionary

```
>>> dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars', 2006):7.1}
>>> x = dict.get(('Cars',2006))
>>> x
```

```
7.1
```

```
>>> x = dict.get(('Toy Story',1995))
>>> x == None
```

```
True
```

```
>>> ('Toy Story',1995) in dict
```

```
False
```

Dictionary: Deleting from a Dictionary

- Use .pop() or del.

```
>>> dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars', 2006):7.1}
>>> dict.pop(('Ghostbusters',2016))
```

```
5.4
```

```
>>>  dict
```

```
{('Cars', 2006): 7.1,
('Ghostbusters', 1984): 7.8}
```

```
>>> del dict[('Cars', 2006)]
```

Dictionary: Iterating over a dictionary

```
>>>  dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars', 2006):7.1}
>>> for i in dict:
…       print(i)
```

```
('Ghostbusters', 2016)
('Cars', 2006)
('Ghostbusters', 1984)
```

Dictionary: Iterating over a dictionary

```
>>>  dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars', 2006):7.1}
>>> for key, value in dict.items():
…       print(key,":",value)
```

```
('Ghostbusters', 2016) : 5.4

('Cars', 2006) : 2.1
('Ghostbusters', 1984) : 7.8
```

Dictionary: Iterating over a dictionary

```
>>>  dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars', 2006):7.1}
>>> for i in dict:
…        dict.pop(i)
```

```
5.4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
RuntimeError: dictionary changed size
during iteration
```

Dictionary: Iterating over a dictionary

- You can remove a pair in a dictionary selectively by the following way:

```
>>>  dict = {('Ghostbusters', 2016): 5.4,
('Ghostbusters', 1984): 7.8, ('Cars',
2006):7.1}
>>> to_remove = [];
>>> for i in dict:
…       if(i[1] < 2000):
…             to_remove.append(i)
>>> for i in to_remove:
...     dict.pop(i)
...
>>> dict
```

```
{('Ghostbusters', 2016): 5.4,
('Cars', 2006): 2.1}
```

Sets: Basics

- Unordered
- Unique (no duplicates)
- Support set operations, e.g., union, and intersection

Sets: Created and Add

```
>>> leos_colors = set(['blue','green','red'])
>>> leos_colors
```

```
{'green', 'red', 'blue'}
```

```
>>> leos_colors.add('yellow')
>>> leos_colors
```

```
{'green', 'red', 'yellow', 'blue'}
```

```
>>> leos_colors.add('blue')
>>> leos_colors
```

```
{'green', 'red', 'yellow', 'blue'}
```

Sets: Discard

- The discard method will fail if the item is not in set.

```
>>> leos_colors = set(['blue','green','red'])
>>> leos_colors
```

```
{'green', 'red', 'blue'}
```

```
>>> leos_colors.discard('green')
>>> leos_colors
```

```
{'red', 'blue'}
```

Sets: Union / Labs

- Union can be done with the | operator, e.g., set1 | set2

```
>>> leos_colors = set(['blue','green','red'])
>>> ilkays_colors = set(['blue','yellow'])
>>> either = ilkays_colors.union(leos_colors)
>>> either
```

```
{'green', 'red', 'yellow', 'blue'}
```

Sets: Intersection / Labs

- Intersection can be done with the & operator, e.g., set1 & set2

```
>>> leos_colors = set(['blue','green','red'])
>>> ilkays_colors = set(['blue','yellow'])
>>> both = ilkays_colors.intersection(leos_colors)
>>> both
```

```
{'blue'}
```